Custom ping-pongs

If your clientside websocket framework doesn't support ping-pong control frames, you can use Talo's custom heartbeat message to keep connections alive.

> **INFO**
>
> The browser's native WebSocket API doesn't expose access to ping/pong control frames - browsers handle them automatically under the hood. If you're using the browser WebSocket API directly, you'll need to use this custom heartbeat mechanism.

Heartbeat message

To ensure your socket connections don't fail their heartbeat checks, send the `v1.heartbeat` string every 30 seconds.

Unlike other Talo socket messages, this should be a plain string and **not** JSON.

How do Talo heartbeats work?

Talo will ping socket connections every 30 seconds. After sending a ping, Talo expects the connection to respond with a pong. If a pong isn't received after 30 seconds, the connection will be terminated.

The process above uses control frames. If you send the `v1.heartbeat` message, Talo will instantly respond with the same message and mark the connection as "still alive".

The client doesn't need to do anything with the message - this process is simply designed to keep the server informed of healthy open connections.

Example usage

Here's a complete example using the browser's WebSocket API:

```
const socket = new WebSocket('wss://api.trytalo.com')

// send heartbeat every 25 seconds (before the 30-second timeout)
const HEARTBEAT_INTERVAL = 25_000
let heartbeatTimer: number
```

```
socket.addEventListener('open', () => {
  console.log('Connected to Talo')

  // start sending heartbeats
  heartbeatTimer = setInterval(() => {
    if (socket.readyState === WebSocket.OPEN) {
      socket.send('v1.heartbeat')
    }
  }, HEARTBEAT_INTERVAL)
})

socket.addEventListener('close', () => {
  clearInterval(heartbeatTimer)
})

socket.addEventListener('message', (event) => {
  // ignore heartbeat responses
  if (event.data === 'v1.heartbeat') {
    return
  }

  // handle your normal Talo messages here
  const message = JSON.parse(event.data)
  console.log(message.res, message.data)
})
```